

TensorRT and Model Notes

TensorRT

- NVIDIA's inference optimizer designed to decrease inference time through a number of methods while maintaining accuracy
- Can be used with TensorFlow, PyTorch, Caffe, and more
- <https://developer.nvidia.com/tensorrt>

TensorRT Lite

- https://docs.nvidia.com/deeplearning/sdk/tensorrt-api/python_api/pkg_ref/lite.html
- Abstracted, high-level python interface for TensorRT
- MUCH easier to use...particularly because the TensorRT documentation isn't great

Preparing Your Model for Inference With TensorRT from TensorFlow

1. Need to "freeze" trained model
 - Combine .meta file (contains the graph structure), .data file (contains weight values), and .index file (identifies checkpoint file) into a protobuf (.pb) file
 - Remove training nodes (e.g. accuracy, loss, etc.)
 - Turns TensorFlow Variables into Constants
 - Also takes in the name of the graph node that will be performing inference (i.e. the output layer)
2. Convert frozen model to uff (universal file format) using uff module installed with TensorRT
 - uff's from_tensorflow function which takes in frozen graph file, and name of output node as well
3. Create uff parser (tensorrt.parsers.uffparser.create_uff_parser())
 - use parser to register the input node with args:
 - name of input node in TF graph
 - size of inputs in CHW format
 - integer representing the input order on which the framework input was originally
 - Still not sure what this does, but for TF use 0
 - use parser to register the output node that will be providing the inference with args:

- name of output node in TF graph

Creating Your TensorRT Engine

1. Pass in the uff model stream and create the TensorRT engine

```
tensorrt.utils.uff_to_trt_engine(logger, stream, parser, max_batch_size,
                                max_workspace_size, datatype=<DataType.FLOAT: 0>, plugin_factory=None,
                                calibrator=None)
```

- where stream is your uff model,
 - parser is the uff parser,
 - datatype can be FP32, FP16, FP64, or INT8
 - max_workspace size is typically around $1 \ll 20$
2. destroy uff parser now that engine is created
 - parser.destroy()
 3. Create a runtime and execution context for the engine
 4. Allocate memory on GPU for inputs and outputs
 - a. for input: max batch size x number of bytes in input images
 - b. for output: max batch size x number of bytes output requires
 5. Allocate memory on the host machine to transfer outputs to
 6. Create pointers to the GPU memory where inputs and outputs are located

Performing Inference With Your TensorRT Engine

1. Create a cuda stream
2. Transfer the inputs to the device TensorRT is performing inference on
3. Execute the model, and return the inference to the host
4. synchronize the threads

Saving Your Engine for Later Use

To save:

```
trt.utils.write_engine_to_file("filename", engine.serialize())
```

To load in a saved engine:

```
trt.utils.load_engine(G_LOGGER, "filename")
```

Useful Examples From TensorRT Documentation

https://docs.nvidia.com/deeplearning/sdk/tensorrt-api/python_api/workflows/tf_to_tensorrt.html

Literature Reivew

Jha, Susmit, et al. "Safe Autonomy under Perception Uncertainty Using Chance-Constrained Temporal Logic." *Journal of Automated Reasoning*, no. 1, 2018, p. 43. EBSCOhost, doi:10.1007/s10817-017-9413-9.

a. **Summary:**

- i. Proposes a model that uses temporal patterns in (sensor) data to help deal with noise and uncertain environments/signals.

b. **Methods:**

- i. Cone programming problem that modifies perception coefficients and state variables.
 - 1. State variables are position of the vehicle, its velocity, acceleration, orientation, angular velocities, etc.
- ii. Half-planes form the basic unit of representation of knowledge acquired through perception.
 - 1. motivated by the observation that perception algorithms often employ half-plane learning techniques such as Bayesian linear regression and classifiers.

c. **Results:**

Ort, Teddy, Liam Paull, and Daniela Rus. "Autonomous Vehicle Navigation in Rural Environments without Detailed Prior Maps." *International Conference on Robotics and Automation*. 2018.

a. **Summary:**

- i. In this paper, we address the problem of autonomous navigation in rural environments through a novel mapless driving framework that combines sparse topological maps for global navigation with a sensor-based perception system for local navigation.

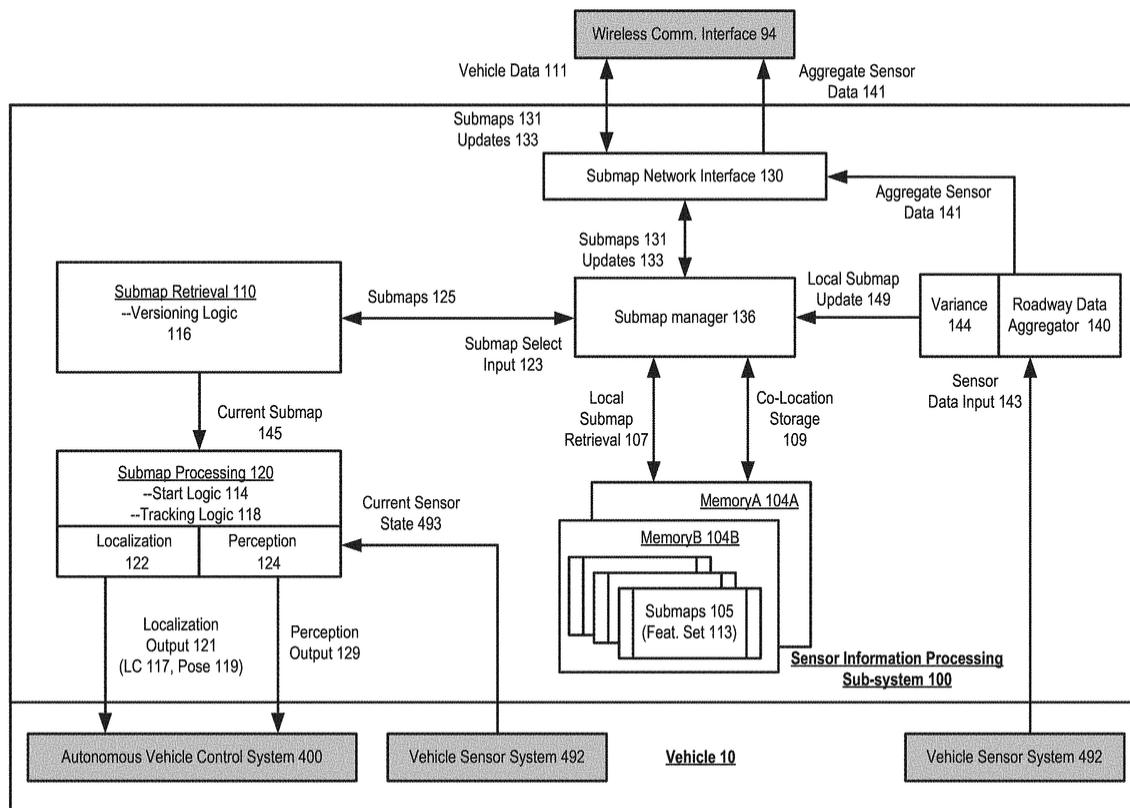
b. **Methods:**

- i. First, a local navigation goal within the sensor view of the vehicle is chosen as a waypoint leading towards the global goal.
- ii. Next, the local perception system generates a feasible trajectory in the vehicle frame to reach the waypoint while abiding by the rules of the road for the segment being traversed.
- iii. These trajectories are updated to remain in the local frame using the vehicle's odometry and the associated uncertainty based on the least-squares residual and a recursive filtering approach

c. Results:

- i. The system was evaluated at a test site in Devens, MA. The roads were single-lane, unmarked roads which do not have curbs or barriers at the boundaries (see Fig. 8). Note that although Fig. 6 shows that the planned trajectories occasionally deviated from the road. These deviations were typically far ahead and since the trajectories are replanned after the vehicle traverses only a short distance as described in V-A, the car was able to reliably follow the 1km path without any need for human intervention or any predefined map.

Milstein, Adam, and Brett Browning. "Autonomous vehicle control using submaps." U.S. Patent Application No. 15/640,289.



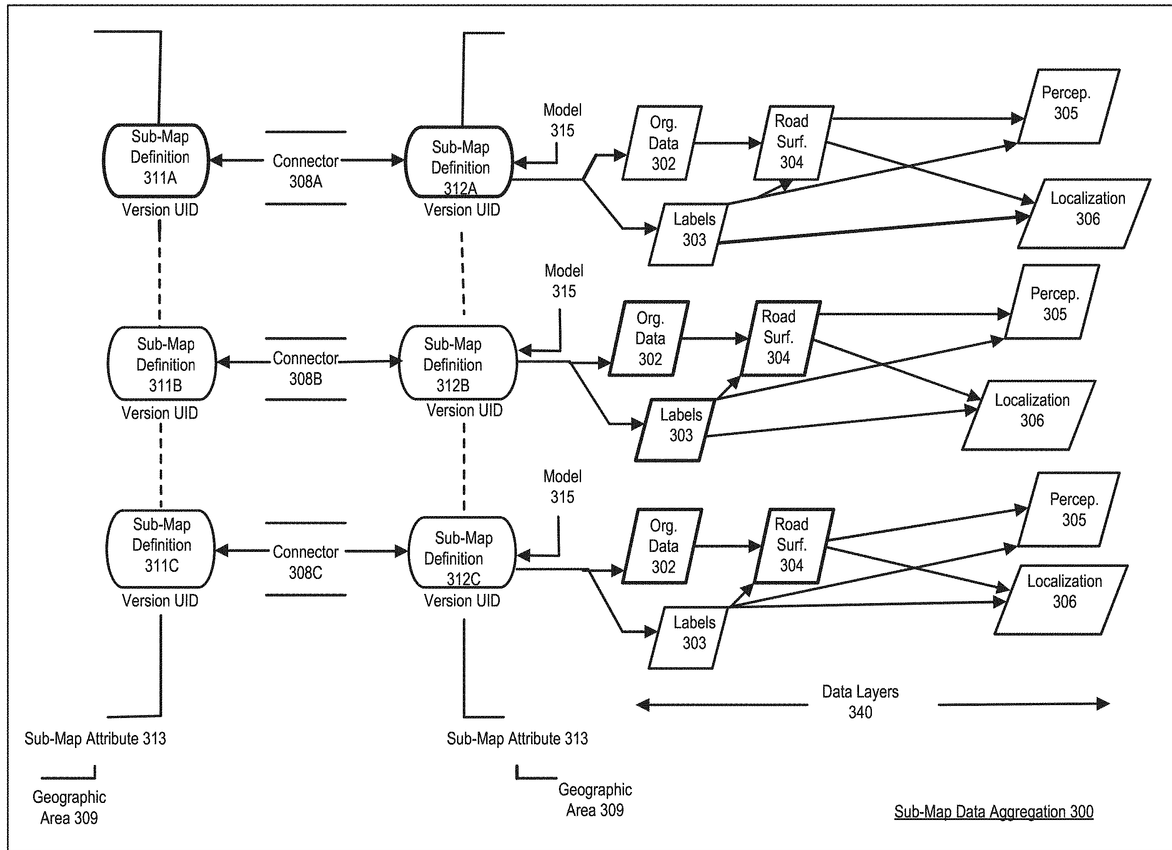


FIG. 3

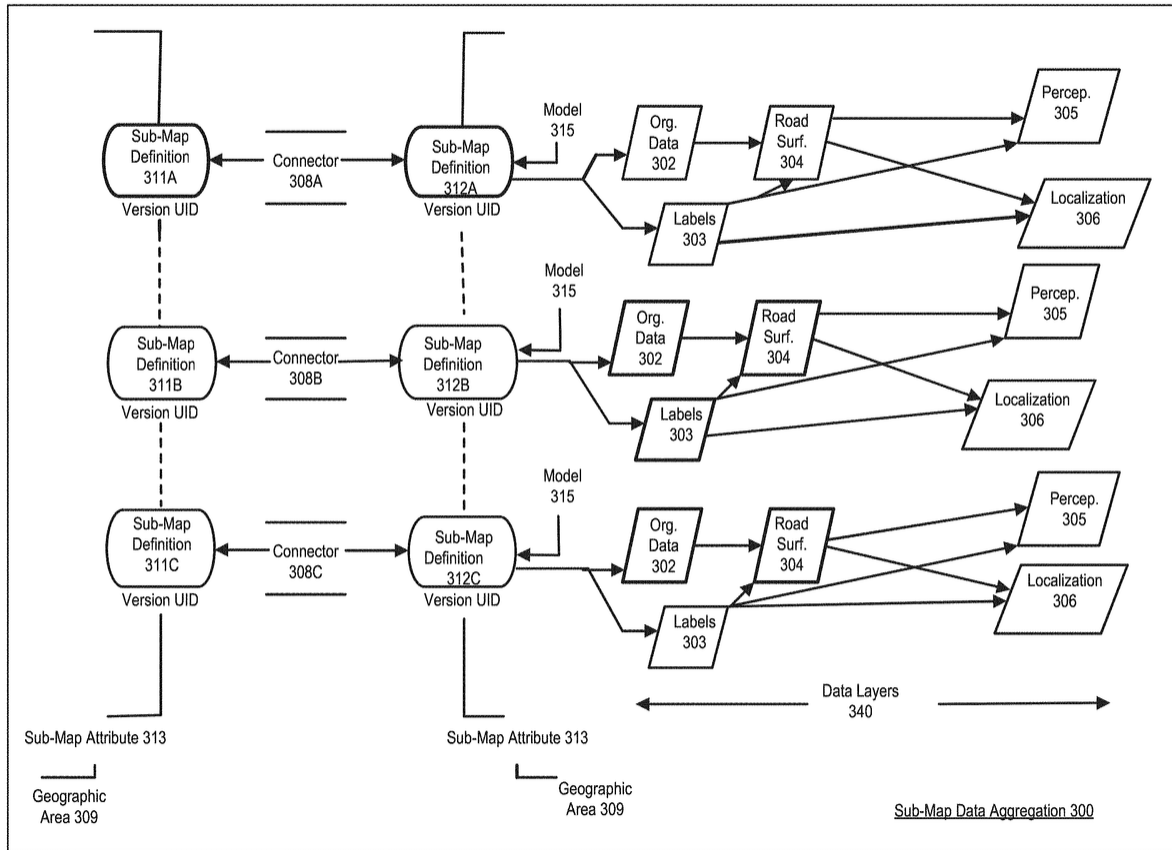


FIG. 3

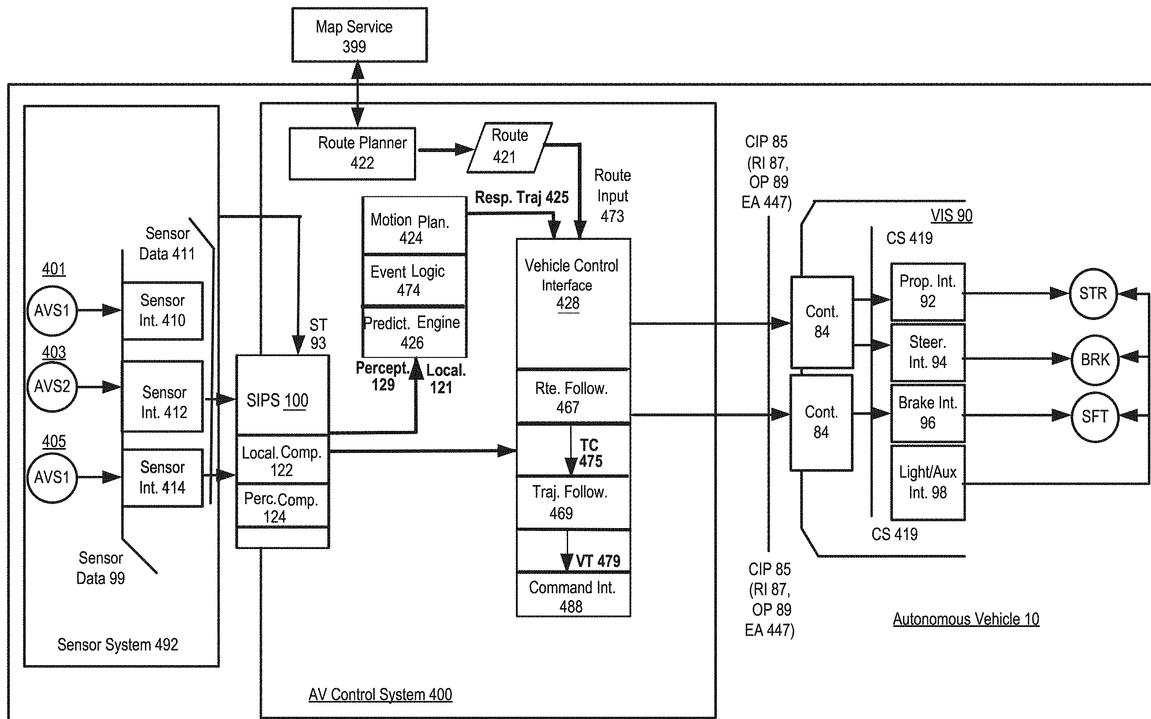


FIG. 4

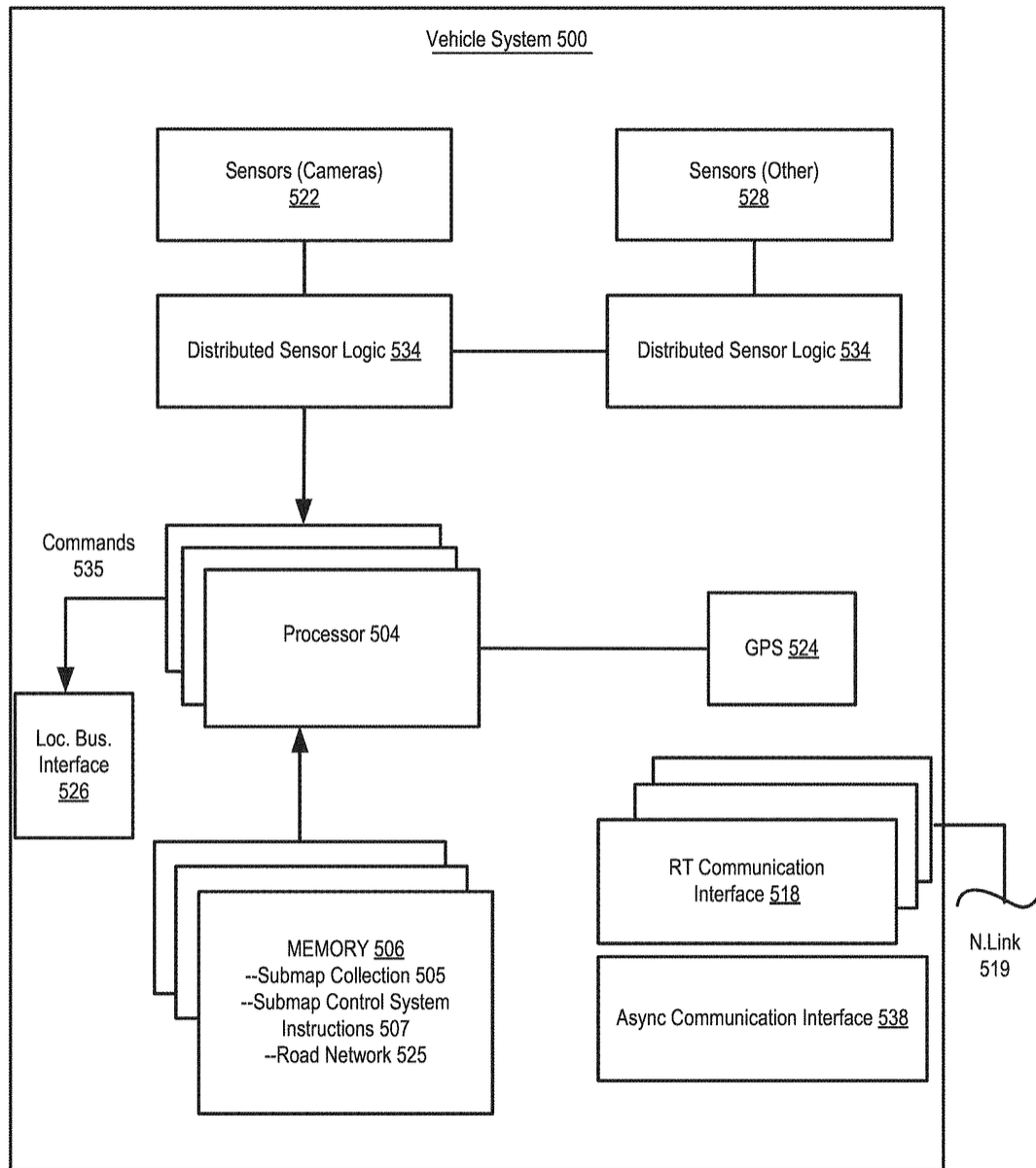


FIG. 5

Fan, Tingxiang, et al. "CrowdMove: Autonomous Mapless Navigation in Crowded Scenarios." *arXiv preprint arXiv:1807.07870* (2018).

<https://youtu.be/iOE6rm8dfBA>

a. Summary:

- i. To learn a robust mapless local planner, we propose a novel 3M (i.e., multi-robot, multi-scenario, and multi-stage) training framework, which exploits a robust policy gradient based reinforcement learning algorithm that is trained in a large-scale robot system in a set of complex environments. We demonstrate that the collision avoidance policy learned from our approach can find collision-free paths for nonholonomic robots and can be generalized effectively to various scenarios and different mobile platforms

b. Methodology:

- i. Used a laser range finder
 - 1. We used the Hokuyo urg-04lx 2D LiDAR as our laser scanner, the Pozyx Ultra-WideBand(UWB) modules as the indoor localization system, and the Nvidia Jetson TX1 as our onboard computer. In this way, the cost of the sensor kit is economical.
- ii. we propose a two-stage training process
 - 1. In the first stage, we only train 20 robots on the random scenarios without any obstacles
 - 2. The policy will continue to be updated in Stage 2, where the number of robots increases to 58, and they are trained on the richer and more complex scenarios

c. Results:

- i. Vehicles were able to navigate through crowded, unseen areas without accident, even though they were different sizes, speeds, hardware configurations, etc.

3) **Reward design:** Our objective is to avoid collisions during navigation and minimize the arrival time. A reward function is designed to guide robots to achieve this objective:

$$r^t = (g_r)^t + (c_r)^t + (w_r)^t. \quad (1)$$

The reward r received at timestep t is a sum of three terms, g_r , c_r , and w_r . In particular, the robot is awarded by $(g_r)^t$ for reaching its goal:

$$(g_r)^t = \begin{cases} r_{arrival} & \text{if } \|\mathbf{p}^t - \mathbf{g}\| < 0.1 \\ \omega_g(\|\mathbf{p}^{t-1} - \mathbf{g}\| - \|\mathbf{p}^t - \mathbf{g}\|) & \text{otherwise.} \end{cases} \quad (2)$$

When the robot collides with other robots or obstacles in the environment, it is penalized by $(c_r)^t$:

$$(c_r)^t = \begin{cases} r_{collision} & \text{if } collision \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

To encourage the robot to move smoothly, a small penalty $(w_r)^t$ is introduced to punish the large rotational velocities:

$$(w_r)^t = \omega_w |w^t| \quad \text{if } |w^t| > 0.7. \quad (4)$$

We set $r_{arrival} = 15$, $\omega_g = 2.5$, $r_{collision} = -15$ and $\omega_w = -0.1$ in the training procedure.

Kendall, Alex, et al. "Learning to Drive in a Day." *arXiv preprint arXiv:1807.00412* (2018).

a. **Summary:**

- i. We demonstrate the first application of deep reinforcement learning to autonomous driving. From randomly initialised parameters, our model is

able to learn a policy for lane following in a handful of training episodes using a single monocular image as input. We provide a general and easy to obtain reward: the distance travelled by the vehicle without the safety driver taking control. We use a continuous, model-free deep reinforcement learning algorithm, with all exploration and optimisation performed on-vehicle.

b. Methods:

- i. A key focus of this paper is the set-up of driving as an MDP. Two methods of describing state are evaluated.
 1. Using the single, monocular image itself
 2. Using a compressed version of the image (ie coefficients from a VAE)
- ii. Driving itself has what one might think are a natural set of actions: throttle, brake, signals etc. But what domain should the output of the reinforcement learning algorithm be? The throttle itself can be described as discrete, either on or off, or continuous, in a range isometric to $[0, 1]$. An alternative is to reparameterise the throttle in terms of a speed set-point, with throttle output by a classical controller in an attempt to match the set-point. Overall, experiments on a simple simulator (Section 4.1) showed that continuous actions, whilst somewhat harder to learn, provide for a smoother controller. We found it important to limit the steering angle requested from the vehicle.
- iii. We selected a simple continuous action domain model-free reinforcement learning algorithm: deep deterministic policy gradients

c. Results:

- i. Simulations and real-world driving proved successful
- ii. VAE was most efficient, taking least time to learn
- iii. Raw image was also able to learn, however.